

AEON OS

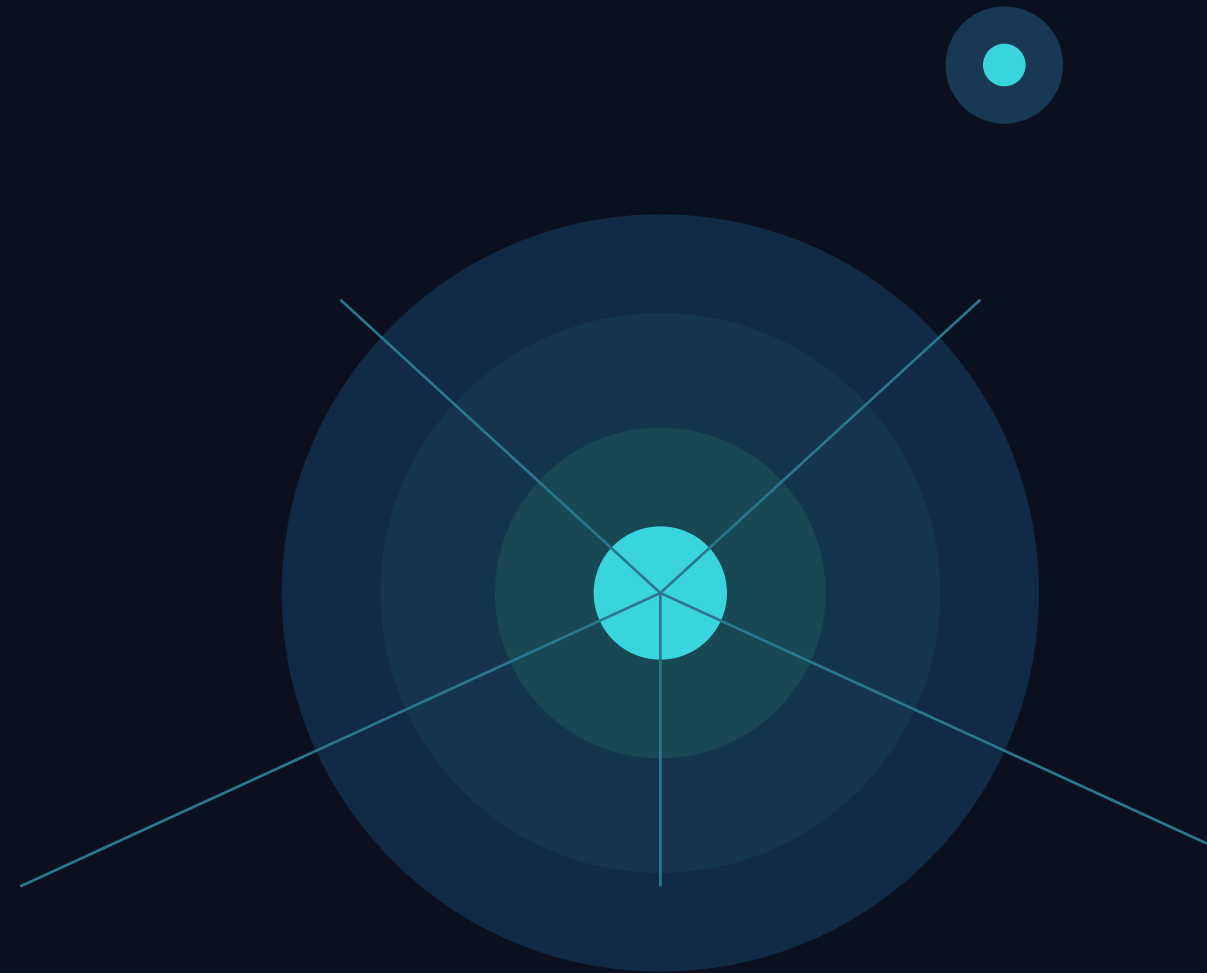
Advanced Evolutionary Orchestrator Network

Project architecture, implemented capabilities,
runtime flow, and production readiness

REPOSITORY SNAPSHOT

beatznlg/aeon

August 4, 2026



01 | What has been built

AEON is an AI operating layer for multi-tenant workspaces, not only a chat screen.

Product surface

Next.js 14 dashboard with protected OS, chat, settings, admin, monitoring, automation, RAG, swarm, billing, and sector surfaces.

Flask/Python kernel for auth, agent execution, tools, workflows, automations, integrations, health, metrics, and governance.

OpenAPI documentation plus Python, TypeScript, and Go client paths for programmatic use.

Runtime capabilities

Pluggable LLM providers: stub, OpenAI, Anthropic, Ollama, Hugging Face, and local Qwen path.

Persistent memory, goals, reflection, tool execution, RAG retrieval, workflow actions, and multi-agent swarms.

Operational controls: budgets, approvals, cooldowns, audit logs, anomaly detection, incident models, backups, DR, and SIEM exports.

02 | Repository at a glance

Measured from the repository snapshot used for this presentation.

Frontend

36 page surfaces
99 Next.js API route files
React + TypeScript + Tailwind
Recharts, Framer Motion, NextAuth

Backend

41 root Python modules
Flask HTTP kernel
SQLAlchemy models
Supabase REST integrations
Celery/Redis option

Delivery

22 test files
18 Supabase migrations
Docker Compose stack
Railway + Vercel deployment guide
OpenAPI + SDKs

03 | System topology

The intended production split keeps long-running AI and automation work off the frontend host.

Request plane

Browser -> Next.js dashboard and server-side proxy routes.

NextAuth session and JWT claims establish identity, role, and selected workspace.

Next.js forwards protected operations to Flask through `AEON_PYTHON_URL`.

Flask returns workspace-scoped JSON and operational status.

Execution plane

Flask -> ReflectiveAgent / AeonKernel -> selected LLM provider.

SQLAlchemy/Postgres stores identity, governance, automation, incident, DR, and SIEM state.

AEON_ROOT stores agent files, memory, goals, skills, ledgers, and local vector indexes.

Redis/Celery provides optional distributed automation dispatch and scheduled work.

04 | The most important request flow

A single chat or domain request crosses several policy and persistence boundaries.

Browser to kernel

1. User enters chat or app prompt in the dashboard.
2. Next.js route checks session and resolves workspace context.
3. Proxy forwards method, JSON body, bearer context, and workspace headers.
4. Flask decorators enforce authentication and workspace role.
5. Kernel chooses the workspace agent and provider.

Kernel to result

6. Agent builds context from memory, goals, RAG, and tool registry.
7. LLM response may emit structured tool calls.
8. Safe runner validates and executes allowed tools.
9. Memory, vitals, audit, metrics, and usage are updated.
10. JSON result returns to the UI with backend and execution metadata.

05 | Frontend product experience

The web application organizes AEON by operator task and business sector.

Workspace OS

- Dashboard overview
- Chat and history
- App/sector launcher
- Automations
- Workflows
- Approvals

Operations

- Monitoring
- Observability
- Incidents
- Anomalies
- Audit/governance
- Disaster recovery
- SIEM

Control plane

- Settings
- API keys
- Billing
- Integrations
- SSO/SCIM
- Admin panel
- Notifications

06 | Backend kernel modules

The root Python modules are intentionally separated by responsibility, while `aeon_server.py` composes the HTTP surface.

Core AI

`aeon.py`: AeonKernel, ReflectiveAgent, MemoryBundle, GoalState, SelfModel, CodeSandbox, CodeEvolver, tools, ledger, and service registry.

`aeon_os.py`: workspace-scoped agent lifecycle and orchestration.

`aeon_llm.py`: provider abstraction and provider-specific adapters.

`aeon_chat.py`: chat/session helpers.

`aeon_rag.py` + `aeon_vector_store.py`: ingestion, chunking, embeddings, indexing, hybrid retrieval.

Platform services

`aeon_db.py`: SQLAlchemy schema and persistence helpers.

`aeon_auth.py`: JWT/API-key auth and workspace authorization.

`aeon_automations.py`: event rules, chains, schedules, budgets, approvals, and resumptions.

`aeon_worker.py`: Celery bridge with eager/inline fallback.

`aeon_integrations.py`, `aeon_security.py`, `aeon_anomalies.py`, `aeon_siem.py`, and DR modules.

07 | Tenancy, identity, and authorization

Every sensitive capability is designed around a current user and workspace context.

Identity model

Users belong to tenants and can have memberships in multiple workspaces.

Workspace membership carries VIEWER, OPERATOR, ADMIN, or SUPER_ADMIN-style access.

JWT access tokens and hashed API keys support human and machine callers.

Refresh-token storage and JWT rotation endpoints support credential lifecycle management.

Enforcement

`require_auth` protects authenticated routes.

Workspace access and role decorators guard data-plane mutations.

Frontend middleware protects `/os`, `/settings`, and `/chat` and redirects unauthorized admin routes.

Audit events retain user, module, workspace, action, metadata, and PII-redaction status.

08 | Relational data model

SQLAlchemy models mirror the durable control-plane entities; Supabase migrations extend the same domain.

Identity

- tenants
- users
- workspaces
- memberships
- refresh_tokens
- identity_links

Automation

- automation_executions
- automation_policies
- automation_budgets
- rule snapshots
- variables
- approvals

Enterprise ops

- audit_logs
- SSO providers
- SCIM tokens
- security config
- anomalies
- incidents
- runbooks
- backups
- DR
- SIEM

09 | Filesystem state and memory persistence

AEON uses files for agent-local state and SQL for shared governance and operational records.

AEON_ROOT state

Agent directories hold state.json and other per-agent artifacts.

MemoryBundle maintains episodic events, semantic facts/links, and procedural skill metadata.

GoalState persists objectives in goals/goals.jsonl and reloads them across restarts.

Ledgers, bounty state, evolved skills, and local vector indexes are also file-oriented in the local backend.

Durability boundary

Local filesystem is useful for development and a mounted production volume.

Serverless /tmp or in-memory fallback is ephemeral and cannot be the source of truth for durable operations.

Production should mount AEON_ROOT and use Postgres/Supabase for shared records.

Storage abstraction supports local and Supabase-backed object/file storage.

10 | Agent loop, memory, and reflection

The reflective agent wraps a tool loop with durable context and self-observation.

Tick lifecycle

Receive query and build a system prompt with available tools.

Generate with the selected LLM provider.

Parse structured tool-call JSON and execute through the safe runner.

Replace tool calls with bounded result summaries in the answer.

Record user/bot events, tool outcomes, causal credit, and self-model vitals.

Reflection layer

MemoryBundle supplies recent context and semantic facts.

GoalState selects the highest-priority open objective.

SelfModel tracks uptime, ticks, calls, errors, success rate, and wellbeing.

Decision logic can tick, reduce error, or trigger controlled tool evolution after poor performance.

CodeEvolver validates candidate tools before registration; evolution suggestions are not auto-executed.

11 | Tools, safety, and model providers

Provider choice is configuration-driven; tool execution is deliberately more constrained than text generation.

Providers

stub for local boot
OpenAI
Anthropic
Ollama
Hugging Face inference
Qwen local path

Tool surface

Math
search/fetch
GitHub search
API catalog/fetch
skills
bounties
service quotes

Safety gates

AST/code analysis
blocked imports/calls
HTTP scheme validation
wallet whitelist
broadcast disabled by default

12 | RAG and knowledge retrieval

Knowledge bases extend agent context without requiring every document to fit in a prompt.

Ingestion path

Documents are accepted through the RAG API and normalized into chunks.

Chunk metadata preserves document/workspace identity and retrieval context.

Embeddings use sentence-transformers when available, with a deterministic local fallback path.

Vector storage can be local/disk-backed or backed by Supabase/pgvector-compatible infrastructure.

Search path

Query is embedded and/or keyword-indexed.

Hybrid scoring combines lexical and semantic relevance.

Top chunks are returned to the agent or chat route as grounded context.

Operational requirement: durable document storage and vector persistence must be configured for production.

13 | Automation engine

AEON automations behave like governed event-driven workflows rather than one-shot webhooks.

Trigger and policy

Rules match event types and nested conditions with operators such as `$gt`, `$in`, `$contains`, `$regex`, `$and`/`$or`/`$not`.

Cooldowns prevent runaway repetition; workspace/rule budgets can warn or block.

Approval-required rules pause at a human-in-the-loop checkpoint.

Cron scheduling and inbound webhooks create external and time-based triggers.

Execution and state

Actions support sequential chains, conditional `run_if`, loops, fallbacks, `continue_on_error`, delays, `wait_for_event`, variables, sub-rules, transforms, and parallel branches.

Execution history captures status, result, errors, and runtime metadata.

Sleeping executions persist pending step state and resume later.

Dry-run mode simulates side effects while evaluating logic and transformations.

14 | Automation lifecycle

This is the control path for a typical rule execution.

1. Evaluate

- Fetch workspace rules
- Match event
- Apply condition
- Check cooldown
- Check budget

2. Govern

- Create approval request if required
- Notify operators
- Write audit/execution event
- Persist pending state

3. Execute

- Dispatch Celery when configured
- Otherwise run inline
- Run action chain
- Retry/fallback/resume
- Record final status

15 | Multi-agent swarms

SwarmManager coordinates specialized agents around a shared prompt and task graph.

Roles and planning

Planner decomposes the prompt into SwarmTask records.

Executor agents work assigned tasks by capability.

Reviewer agents inspect outputs and can request corrective work.

Summarizer agents synthesize the final answer.

Coordination contract

SwarmMessage provides sender, recipient, type, timestamp, and swarm identity.

Inbox/outbox and broadcast methods form a lightweight message bus.

Status and messages endpoints expose the run to the dashboard.

Tool-improvement suggestions are returned for explicit human review, never silently applied.

16 | Integrations and approvals

The platform is designed to connect outside systems while keeping sensitive actions governed.

Inbound

- Workspace-created webhook tokens
- External event acceptance
- Automation event logging
- Integration receive routes

Outbound

- Webhook actions
- Slack notifications
- Slack interactive approvals
- Catalog/connector registry
- GitHub-facing tools

Human control

- Pending approval queue
- Operator resolve endpoint
- Approved deferred execution
- Rejected action audit trail

17 | Security and enterprise governance

The codebase includes security controls across HTTP, data handling, identity, and operations.

Security controls

Configurable CORS and baseline security headers including CSP, frame, content-type, and permissions policies.

PII/PHI scanner and recursive metadata sanitization for email, phone, SSN, MRN, API key, and token patterns.

Hashed API keys, rotation endpoints, JWT rotation/status, and workspace RBAC.

Slack HMAC signature verification and restricted webhook behavior.

Enterprise controls

OIDC provider CRUD and SAML degradation path are represented in SSO modules/tests.

SCIM token models support provisioning integration.

Audit logging includes redacted export behavior.

SIEM integrations model provider endpoints, filters, batches, delivery status, and retry logs.

18 | Observability, incidents, and recovery



The latest pushed change added workspace-scoped operations monitoring and API documentation.

Live operations

Health, liveness, readiness, detailed health, Prometheus /metrics, and operations snapshot routes.

Operations snapshot reports readiness, agent vitals, memory/goal counts, worker capacity, and automation status counts without returning prompts or credentials.

Monitoring and observability dashboard pages consume Next.js proxy routes.

Anomaly detection covers automation failure spikes, execution volume, and audit volume.

Resilience domain

Incident, runbook, assignee, severity, and resolution models exist in the persistence layer.

Backup policy, backup job, restore job, DR plan, and DR drill models capture recovery intent.

Docker Compose includes persistent Postgres, Redis, and AEON state volumes.

Operational caveat: durable recovery still requires configured storage, scheduling, and tested runbooks.

19 | Developer experience and API surface



The repository is intended to be operated by humans and integrated by other software.

API contract

docs/openapi.json documents health, auth, workspaces, apps, workflows, swarm, API keys, billing, integrations, RAG, and governance tags.

Swagger-style documentation is served by the backend docs route when enabled.

Workspace-scoped APIs use bearer security and explicit path/query identifiers.

The latest operations snapshot contract documents redaction and access responses.

SDKs and examples

Python client exposes health, chat, and API request helpers with structured `AeonError` behavior.

TypeScript client provides typed HTTP access for browser/server consumers.

Go client and SDK generator are included in the repository structure.

examples/ contains runnable integration starting points.

20 | Deployment and runtime modes

The repository supports a local all-in-one path, split production hosting, and an experimental serverless adapter.

Recommended production

Vercel -> Next.js frontend with NEXTAUTH_SECRET, NEXTAUTH_URL, and AEON_PYTHON_URL.

Railway or Docker -> long-running Flask backend with Postgres and mounted AEON_ROOT.

Redis + Celery worker/beat for durable distributed automation dispatch.

Prometheus/Grafana/Alertmanager for production telemetry.

Local and experimental

docker compose up --build starts Postgres, Redis, Flask, Celery worker/beat, and Next.js.

web/package.json offers dev, dev:full, build, start, lint, and formatting scripts.

Vercel Python backend exists for simple requests but is not recommended for long jobs or heavy ML.

Stub LLM provider is the lowest-friction first boot; external provider keys are optional until needed.

21 | Configuration checklist

Names below are configuration categories, not secrets or values.

Required core

NEXTAUTH_SECRET
NEXTAUTH_URL
AEON_DATABASE_URL
AEON_PYTHON_URL
AEON_ROOT

AI and data

AEON_LLM_PROVIDER
OPENAI_API_KEY
ANTHROPIC_API_KEY
HUGGINGFACE_TOKEN
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY

Ops and commerce

AEON_REDIS_URL
AEON_WORKER_THREADS
STRIPE_API_KEY
STRIPE_WEBHOOK_SECRET
CORS/HSTS settings
Slack signing secret

22 | Verification and current caveats

This presentation describes repository capabilities; external services still need environment-specific validation.

What is verified in code

22 Python test files cover auth, storage, security, anomalies, swarm, worker dispatch, metrics, SSO, SDK, and governance areas.

18 Supabase migration files extend automation, variables, snapshots, analytics, and enterprise domains.

The observability change was pushed as commit 34d4d05 and main matched origin/main at the snapshot.

The repository working tree was clean before presentation generation.

What still needs an environment

Freebuff preview/deploy CLIs were not available in the earlier shell, so preview readiness was not claimed.

Postgres/Supabase, Redis, provider keys, Stripe, Slack, SSO, and persistent volumes require operator setup.

Serverless filesystem, in-memory caches, thread pools, and local fallbacks are not durable production substitutes.

Sector data generators contain deterministic demo-style values; connect real data sources before regulated use.

23 | Roadmap from the current foundation

The platform has a broad control plane; the next work should deepen reliability and product proof.

Highest-value hardening

Run the complete test and typecheck gates in a fully provisioned environment.

Exercise every frontend route against a live backend and seeded workspace.

Add end-to-end automation tests covering approval, resume, parallel, retry, and budget paths.

Make migration execution and deployment health checks deterministic across Postgres/Supabase.

Scale and product depth

Replace process-local scheduling with a single durable scheduler topology in production.

Add formal tenant isolation tests and policy tests for every sensitive route.

Connect sector modules to real customer data contracts instead of generated sector samples.

Publish performance, cost, latency, and recovery objectives backed by measured runs.